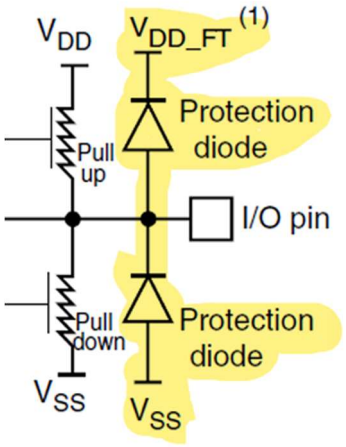
02) GPIO e Interrupts

General Purpose Input/Output (GPIO)

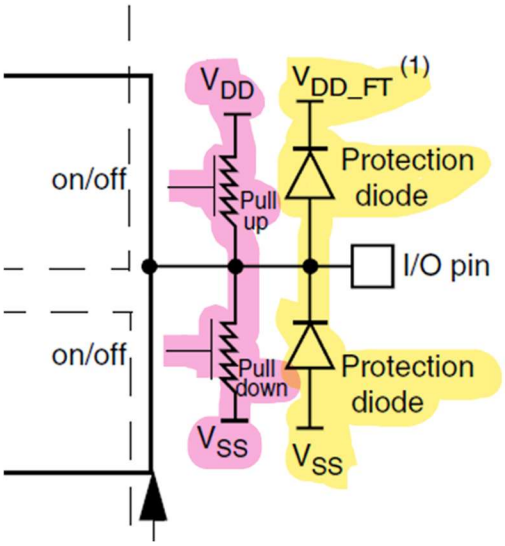
GPIO (General-Purpose Input/Output): sono pin del microcontrollore configurabili come **ingressi o uscite digitali**, utilizzati per comunicare con dispositivi esterni, ad esempio leggere un pulsante o controllare un LED.

- **LED di protezione** presenti all'ingresso di una GPIO servono a proteggere il circuito interno del microcontrollore da **tensioni troppo alte o troppo basse**.
  Supponiamo che la GPIO sia alimentata a **3,3 V**.
  - **Tensione troppo alta:** se sulla GPIO arriva una tensione superiore a VDD, il **diodo superiore** entra in conduzione e devia la corrente verso l'alimentazione, evitando che la tensione interna salga eccessivamente.
  - **Tensione negativa:** se la GPIO viene portata sotto 0 V, il **diodo inferiore** entra in conduzione e devia la corrente verso GND, limitando la tensione negativa.

  In condizioni normali, ad esempio GPIO = **0–3,3 V**, i diodi rimangono praticamente **spenti** e non influenzano il funzionamento del pin.



- I **pull-up** e **pull-down** sono delle **resistenze collegate all'ingresso di una GPIO** per evitare che il pin rimanga in uno stato indefinito quando non è collegato a un segnale.
  - **Pull-up:** La resistenza collega la GPIO a **VDD**:
    - Pulsante **aperto** → la GPIO legge **HIGH (1)**, perché la resistenza la porta verso VDD.
    - Pulsante **chiuso** → la GPIO legge **LOW (0)**, perché il pulsante la collega direttamente a GND.
  - **Pull-down:** La resistenza collega invece la GPIO a **GND**:
    - Pulsante **aperto** → la GPIO legge **LOW (0)**, grazie alla resistenza verso GND.
    - Pulsante **chiuso** → la GPIO legge **HIGH (1)**, perché viene collegata a VDD.

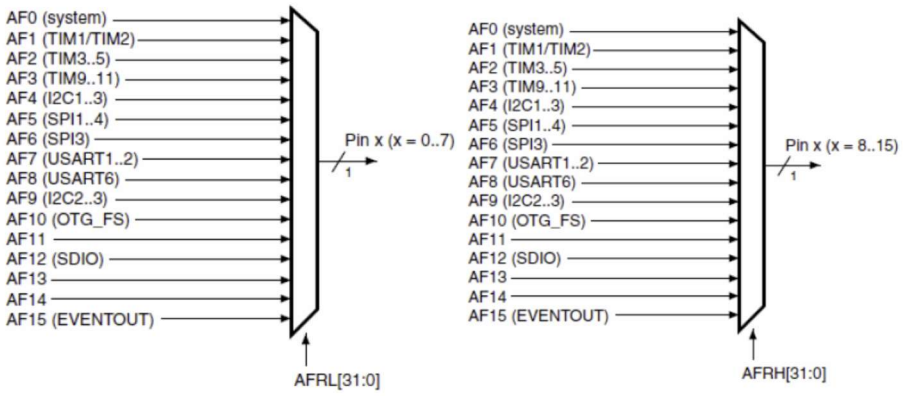


  Senza una pull-up o pull-down, quando l'ingresso non è collegato a nulla, la GPIO è **floating**: il suo valore potrebbe oscillare casualmente tra 0 e 1 a causa di rumore elettrico.
  In sintesi: le pull-up e pull-down **definiscono lo stato logico di default di una GPIO quando non c'è un segnale esterno che la pilota**.

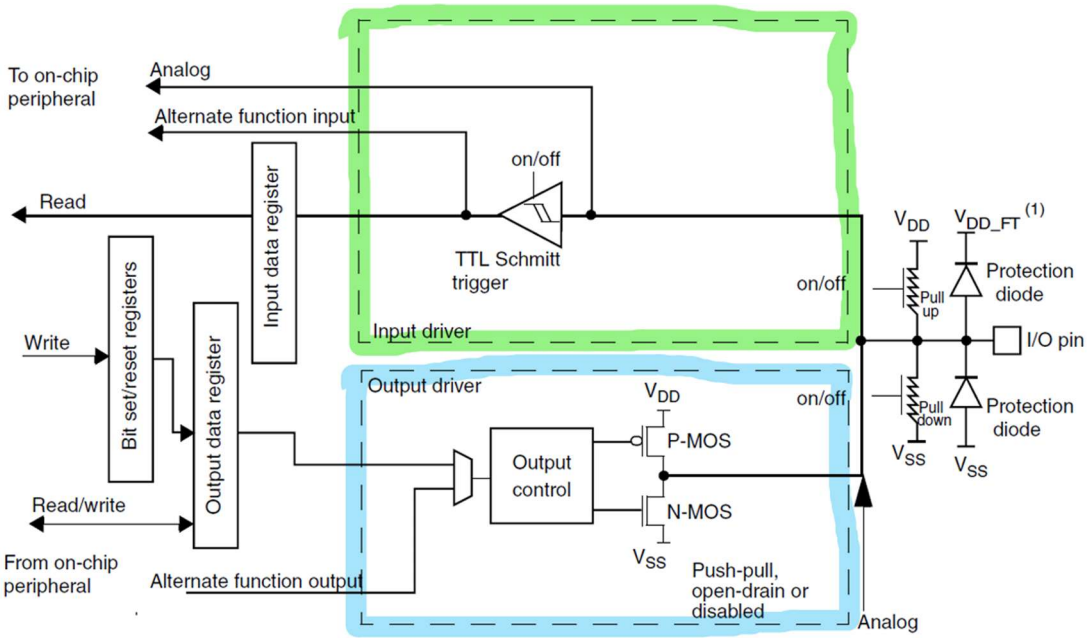
- **Input Driver**: Questa è la parte che ci interessa quando usiamo la GPIO come **ingresso digitale**. Il segnale dal pin passa attraverso il **TTL Schmitt trigger** che serve a trasformare il segnale elettrico proveniente dal pin in un segnale digitale più pulito. Lo **Schmitt trigger** è particolarmente utile perché introduce delle soglie di commutazione e rende il circuito più robusto nei confronti di piccoli disturbi e variazioni del segnale.
  C'è inoltre un controllo **on/off**: quando l'input driver è abilitato, il microcontrollore può **leggere lo stato logico del pin**. Il valore viene memorizzato nell'**Input data register** che può essere letto dalla CPU.

- **Output Driver**: La parte inferiore dello schema è invece il percorso per utilizzare la GPIO come **uscita**. Abbiamo due transistor:
    - **P-MOS: attivo se voglio scrivere 1**
    - **N-MOS: attivo se voglio scrivere 0**

  che costituiscono essenzialmente un **push-pull driver**. L'**Output Data Register** memorizza il valore.
- **Alternate Function Input e Output**: La GPIO non deve necessariamente essere utilizzata come semplice GPIO. Il pin può essere collegato internamente a una **periferica alternativa**:
    - **UART**
    - **SPI**
    - **I²C**
    - **Timer**
    - **PWM**



- **Analog:** Questo permette di utilizzare il pin come ingresso analogico, per esempio collegandolo a un ADC. In questo caso il normale input digitale può essere disabilitato: il circuito digitale di ingresso non deve necessariamente rimanere attivo quando stiamo effettuando una conversione analogica.



Modalità P/N-MOS nell'Output Driver

Modalità	P-MOS	N-MOS	HIGH	LOW	Stato del pin	A cosa serve nella GPIO
Push-pull	ON/OFF	OFF/ON	☑ Attivo	☑ Attivo	HIGH o LOW	Per pilotare normalmente un'uscita digitale, ad esempio **LED, attuatori o segnali digitali**
Open-drain	✗ OFF	ON/OFF	⚠ Tramite pull-up	☑ Attivo	LOW o HIGH con pull-up	Per **condividere una linea** tra più dispositivi, ad esempio nel **bus I²C**, oppure per interfacciarsi con segnali a tensioni diverse
Disabled (Hi-Z)	✗ OFF	✗ OFF	✗	✗	**Alta impedenza**	Per **scollegare il driver dall'uscita**, ad esempio quando il pin viene usato come **ingresso** o quando un altro dispositivo deve controllare la linea

External interrupt (EXTI) peripheral

Interrupt (interruzione): è un meccanismo che permette al microcontrollore di **interrompere temporaneamente l'esecuzione del programma principale** in risposta a un evento, come un cambiamento su un GPIO, per eseguire una specifica routine di gestione dell'interruzione (ISR).

La **EXTI (External Interrupt/Event Controller)** è una periferica del microcontrollore che permette di **rilevare automaticamente un cambiamento su un segnale esterno**, tipicamente collegato a un **GPIO**, e di generare un **interrupt** verso la CPU.
In pratica, permette al microcontrollore di dire: **"Quando succede qualcosa su questo pin, interrompi quello che stai facendo e gestisci l'evento."**
La CPU può quindi eseguire una **Interrupt Service Routine (ISR)**.

Generalmente la EXTI può essere configurata per reagire a:

Trigger	Significato
Rising edge	0 → 1
Falling edge	1 → 0
Both edges	0 → 1 **oppure** 1 → 0

Perché è utile?
Senza interrupt, la CPU dovrebbe continuamente controllare il GPIO:

```
while (1) {
    if (GPIO == 1) {
        // qualcosa è successo
    }
}
```

Questo si chiama **polling** e spreca tempo della CPU. Con EXTI, invece la CPU viene quindi coinvolta **solo quando si verifica l'evento**.

NVIC (Nested Vectored Interrupt Controller)

Il **NVIC (Nested Vectored Interrupt Controller)** è il componente del microcontrollore che **gestisce gli interrupt e decide quale interrupt deve essere servito dalla CPU**.
In particolare, il NVIC:

- riceve le richieste di **interrupt** dalle periferiche, come EXTI, timer, UART, ecc.;
- assegna una **priorità** agli interrupt;
- permette a un interrupt più prioritario di **interrompere un interrupt meno prioritario** (*nested interrupt*);
- indirizza la CPU alla corretta **Interrupt Service Routine (ISR)**.

EXTI rileva l'evento, mentre **NVIC gestisce l'interrupt e stabilisce quando la CPU deve eseguirlo**.

Interrupt vs Event

La differenza fondamentale è **cosa succede alla CPU** quando si verifica l'evento.

Interrupt	Event
L'evento interrompe la CPU	L'evento **non interrompe la CPU**
Viene inviata una richiesta al NVIC	Non necessariamente passa dal NVIC
La CPU sospende temporaneamente il programma	La CPU continua normalmente l'esecuzione
Viene eseguita una ISR (Interrupt Service Routine)	Può attivare direttamente un'altra periferica o un meccanismo hardware
Esempio: premere un pulsante e eseguire una funzione	Esempio: un timer genera un segnale che attiva una periferica

La **EXTI** può generare sia **interrupt** sia **eventi**:

- **Interrupt (relativa al SW, la CPU si ferma, deve eseguire codice):** "È successo qualcosa → **CPU, occupatene!**"
- **Event (relativa all'HW, la CPU non si ferma, non esegue codice):** "È successo qualcosa → **attiva questa funzione hardware**, ma non serve interrompere la CPU."

Quindi, in una frase: **un interrupt coinvolge la CPU tramite il sistema di interrupt; un event permette di propagare un evento hardware senza necessariamente coinvolgere la CPU.**

Esempio:

Esempio di interrupt

Immaginiamo di avere un **pulsante collegato a un GPIO**. Normalmente il microcontrollore sta eseguendo il suo programma. Quando premiamo il pulsante, il segnale sul GPIO cambia, ad esempio da 1 a 0. La **EXTI** si accorge di questo cambiamento e genera un **interrupt**. La richiesta arriva al **NVIC**, che avvisa la CPU. A questo punto la CPU sospende momentaneamente quello che stava facendo e passa ad eseguire una specifica funzione, chiamata **ISR (Interrupt Service Routine)**, che può ad esempio accendere un LED.

Esempio di event

Immaginiamo invece di avere un **timer** che deve avviare una conversione di un **ADC** ogni volta che raggiunge un determinato valore. Quando il timer raggiunge quel valore, può generare un **event** che viene

inviato direttamente all'ADC. L'ADC inizia quindi la conversione senza che la CPU debba essere interrotta ed eseguire una ISR. Mentre l'ADC lavora, la CPU può continuare tranquillamente ad eseguire il programma. In questo caso l'event serve quindi per dire ad un'altra periferica: **"È successo qualcosa, esegui questa operazione"**, senza necessariamente coinvolgere la CPU.

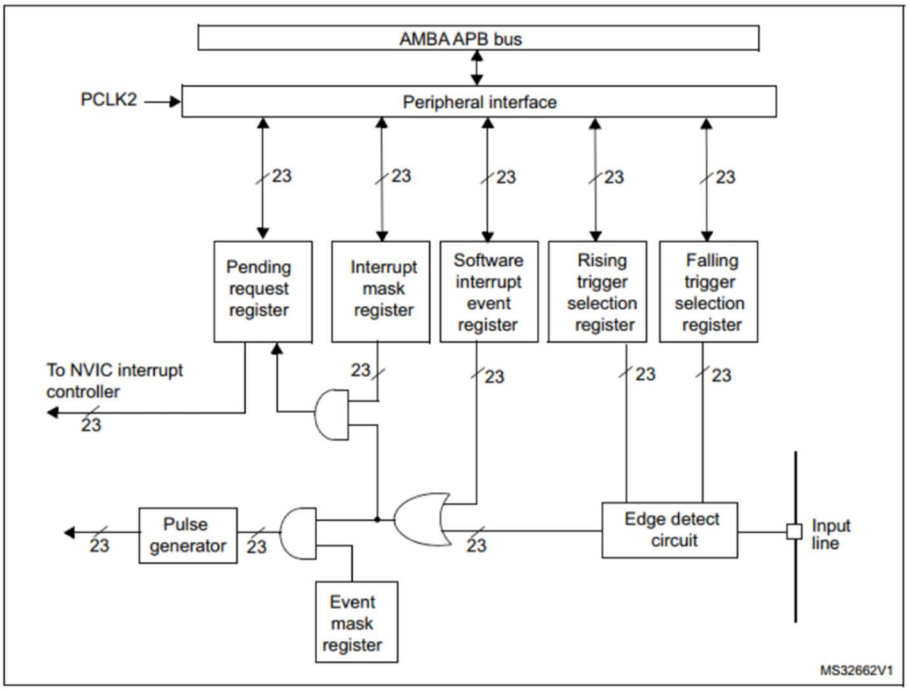
La differenza principale è questa:

- **Interrupt → coinvolge la CPU e fa eseguire del codice.**
- **Event → permette alle periferiche hardware di reagire ad un evento senza necessariamente coinvolgere la CPU.**

Per questo gli **event** sono molto utili nei microcontrollori: permettono di creare collegamenti **periferica → periferica** molto rapidi ed efficienti, riducendo il lavoro della CPU.

Schema Interrupt

La input line va ad un Edge Detect Circuit, che può attivare il Rising oppure il Falling Trigger (Rising/Falling Edge). Quello che succede potrebbe essere un Interrupt oppure un Event: viene decretato attraverso una maschera. Il Pending Request Register alza una flag quando succede qualcosa, se vogliamo che il sistema sia di nuovo sensibile dobbiamo riportare la flag a 0.



Interrupt vector table

La **Interrupt Vector Table** è una la tabella che permette al microcontrollore di sapere **quale funzione eseguire quando si verifica un determinato interrupt**. La **vector table** è una tabella, memorizzata in una zona di memoria del microcontrollore, che contiene una serie di **indirizzi di memoria**. Ogni elemento della tabella è un **puntatore** all'indirizzo iniziale di una specifica **Interrupt Service Routine (ISR)**.

Supponiamo che si verifichi un **interrupt generato dalla EXTI**. Il processore identifica il tipo di interrupt e utilizza la vector table per trovare l'indirizzo associato. Quindi la CPU **non deve "cercare" nel programma quale funzione eseguire**: la vector table le fornisce direttamente l'indirizzo della routine associata a quell'interrupt.

23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
24	31	settable	TIM1_BRK_TIM9	TIM1 Break interrupt and TIM9 global interrupt	0x0000 00A0
25	32	settable	TIM1_UP_TIM10	TIM1 Update interrupt and TIM10 global interrupt	0x0000 00A4
26	33	settable	TIM1_TRG_COM_TIM11	TIM1 Trigger and Commutation interrupts and TIM11 global interrupt	0x0000 00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000 00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000 00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000 00B8
31	38	settable	I2C1_EV	I^2C1 event interrupt	0x0000 00BC
32	39	settable	I2C1_ER	I^2C1 error interrupt	0x0000 00C0
33	40	settable	I2C2_EV	I^2C2 event interrupt	0x0000 00C4
34	41	settable	I2C2_ER	I^2C2 error interrupt	0x0000 00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000 00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000 00D0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0

La Interrupt Vector Table è una tabella di indirizzi che associa ogni tipo di interrupt alla sua specifica ISR. Quando arriva un interrupt, il processore utilizza questa tabella per trovare l'indirizzo della routine da eseguire.

GPIO Functions

1. HAL_GPIO_ReadPin()

GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)

Serve per **leggere lo stato logico di un GPIO**, quindi sapere se sul pin è presente un HIGH o un LOW.

- GPIOx → indica la **porta** GPIO, ad esempio GPIOA, GPIOB, ecc.
- GPIO_Pin → indica quale **pin** della porta vuoi leggere.
- restituisce → GPIO_PIN_SET oppure GPIO_PIN_RESET.

2. HAL_GPIO_WritePin()

void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx,
uint16_t GPIO_Pin,
GPIO_PinState PinState)

Serve per **impostare lo stato di un GPIO configurato come output**.
Gli ultimi due valori possibili sono:

- GPIO_PIN_SET → porta il pin a **HIGH**
- GPIO_PIN_RESET → porta il pin a **LOW**

3. HAL_GPIO_TogglePin()

void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx,
uint16_t GPIO_Pin)

Serve per **invertire lo stato attuale del GPIO**.

- Se il pin è: HIGH → LOW
- se è: LOW → HIGH

4. HAL_GPIO_EXTI_Callback()

__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

Questa funzione è un po' diversa dalle precedenti perché è collegata agli **interrupt della EXTI**.
Quando un GPIO configurato per generare un interrupt rileva l'evento, viene eseguita la relativa routine di interrupt. Dopo che il flag dell'interrupt è stato gestito/reset, la HAL chiama:
HAL_GPIO_EXTI_Callback(GPIO_Pin);
Il parametro GPIO_Pin indica **quale pin ha generato l'interrupt**.

La funzione è dichiarata __weak, cioè STM32 fornisce una **implementazione predefinita** che può essere **sovrascritta dall'utente**.
Per esempio, nel tuo main.c puoi ridefinirla:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

Ora, quando il **GPIO_PIN_0 genera un interrupt**, viene eseguito il callback e viene invertito lo stato di GPIO_PIN_5.

Progetti

Lab02_1°

Lab02_1a: accendere LD2 mentre il bottone blu è premuto e spegnerlo quando è rilasciato, tramite **Polling**.

File .ioc

Pin

- LD2: PA5 = GPIO_Output
- Blue Button: PC13 = GPIO_Input

Codice

Private define

```
#define PORT_BUTTON GPIOC
#define PIN_BUTTON GPIO_PIN_13
#define PORT_LD2 GPIOA
#define PIN_LD2 GPIO_PIN_5
#define PRESSED GPIO_PIN_RESET
// Il bottone è premuto quando lo stato è LOW (0)
```

While(1)

```
  while (1)
  {
        if(HAL_GPIO_ReadPin(PORT_BUTTON, PIN_BUTTON)==PRESSED){
              // Se il Bottone è premuto impostiamo LD2 a HIGH
              HAL_GPIO_WritePin(PORT_LD2,PIN_LD2,GPIO_PIN_SET);
        }else{
              // Altrimenti impostiamo LD2 a LOW
              HAL_GPIO_WritePin(PORT_LD2,PIN_LD2, GPIO_PIN_RESET);
        }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
```

Lab02_1b

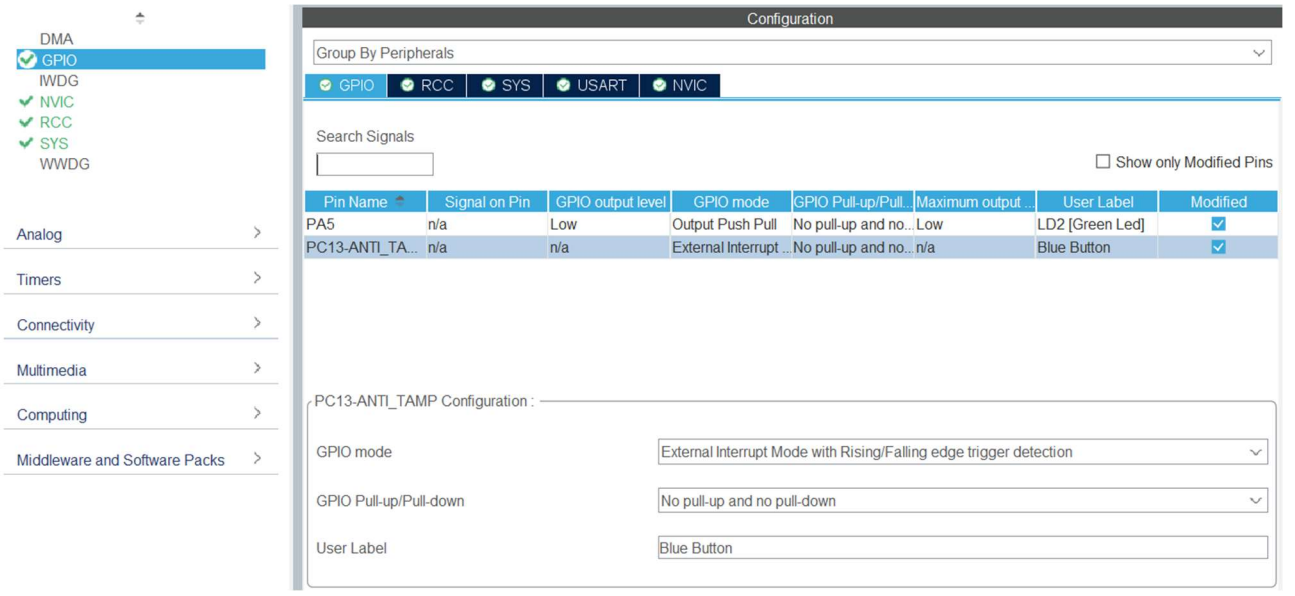
Lab02_1b: accendere LD2 mentre il bottone blu è premuto e spegnerlo quando è rilasciato, tramite **Interrupt**.

File .ioc

Pin

- LD2: PA5 = GPIO_Output
- Blue Button: PC13 = GPIO_EXTI13 (Rising/Falling Detection)

GPIO





Codice

Private Define

```
#define PORT_BUTTON GPIOC
#define PIN_BUTTON GPIO_PIN_13
#define PORT_LD2 GPIOA
#define PIN_LD2 GPIO_PIN_5
#define PRESSED GPIO_PIN_RESET
// Il bottone è premuto quando lo stato è LOW (0)
```

Callback

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == PIN_BUTTON)
    {
      if(HAL_GPIO_ReadPin(PORT_BUTTON, PIN_BUTTON)==PRESSED){
                    // Se il Bottone è premuto impostiamo LD2 a HIGH
```

```
                    HAL_GPIO_WritePin(PORT_LD2,PIN_LD2,GPIO_PIN_SET);
        }else{
                    // Altrimenti impostiamo LD2 a LOW
                    HAL_GPIO_WritePin(PORT_LD2,PIN_LD2, GPIO_PIN_RESET);
        }

    }
}
```